

Hora de tentar sozinhos, agora com KNN

Mesmo dataset mpg, mesma categoria economico que já usaram na prática de Regressão Logística — agora aplicando KNN.

Dessa vez, escolham 2 features (não 1) — e evitem repetir weight + horsepower, que já vimos na teoria.

O objetivo é ter um resultado genuinamente diferente do que foi mostrado em aula, não uma cópia.



Importações necessárias

Incluindo StandardScaler e KNeighborsClassifier, que são novidade em relação à prática anterior.

LEMBRETE DE SINTAXE

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
```



Categoria e 2 features

Recriem a categoria economico, olhem a correlação, e escolham 2 features diferentes de weight+horsepower.

LEMBRETE DE SINTAXE

```
df = pd.read_csv("mpg_desafio_alunos.csv")
df["economico"] = (df["mpg"] >= 23).astype(int)

df.corr(numeric_only=True)["economico"]

X = df[["displacement", "cylinders"]] # troquem pelas 2 escolhidas
y = df["economico"]
```



Tratar valores nulos

Necessário só se uma das features escolhidas for horsepower.

LEMBRETE DE SINTAXE

```
df = df.dropna(subset=["horsepower"])
```



Separar treino e teste

stratify=y de novo, exatamente como na Regressão Logística.

LEMBRETE DE SINTAXE

```
X_train, X_test, y_train, y_test = train_test_split(  
    X, y, test_size=0.2, random_state=5, stratify=y  
)
```

Normalizar as features

Aqui isso NÃO é opcional — sem normalizar, o KNN fica enviesado pela feature de maior escala.

LEMBRETE DE SINTAXE

```
scaler = StandardScaler()  
X_train_norm = scaler.fit_transform(X_train)  
X_test_norm = scaler.transform(X_test)
```

Treinar com K=5 e avaliar

Um primeiro modelo, com um K arbitrário, só pra ver funcionando.

LEMBRETE DE SINTAXE

```
modelo = KNeighborsClassifier(n_neighbors=5)
modelo.fit(X_train_norm, y_train)

pred = modelo.predict(X_test_norm)
print(f"Acuracia com K=5: {accuracy_score(y_test, pred):.2%}")
```

Buscar o melhor K

Testem de 1 a 20, e vejam qual K se sai melhor com a combinação de features que vocês escolheram.

LEMBRETE DE SINTAXE

```
valores_k = range(1, 21)
acuracias = []
for k in valores_k:
    m = KNeighborsClassifier(n_neighbors=k).fit(X_train_norm, y_train)
    acuracias.append(accuracy_score(y_test, m.predict(X_test_norm)))

melhor_k = list(valores_k)[np.argmax(acuracias)]
print(f"Melhor K: {melhor_k}")

plt.plot(list(valores_k), acuracias, marker="o")
plt.axvline(melhor_k, color="red", linestyle="--")
plt.show()
```



Avaliação final

Retreinem com o melhor K encontrado, e rodem no mpg_desafio_final.csv.

LEMBRETE DE SINTAXE

```
modelo_final = KNeighborsClassifier(n_neighbors=melhor_k)
modelo_final.fit(X_train_norm, y_train)

df_final = pd.read_csv("mpg_desafio_final.csv")
df_final["economico"] = (df_final["mpg"] >= 23).astype(int)

X_final = df_final[["displacement", "cylinders"]]
X_final_norm = scaler.transform(X_final)
y_final = df_final["economico"]

pred_final = modelo_final.predict(X_final_norm)
print(f"Acuracia final: {accuracy_score(y_final, pred_final):.2%}")
```



Refletir

Comparando KNN com a Regressão Logística que vocês já praticaram: qual foi mais simples de configurar? Qual deu mais trabalho (normalização, escolha de K)? Os resultados finais ficaram parecidos?

Não existe algoritmo universalmente melhor — cada um tem seu preço e seu ganho.



Pipeline completo, com decisões próprias

Features escolhidas com dado, normalização obrigatória, K encontrado por busca, avaliado no conjunto nunca visto — tudo com decisões de vocês.

Fim da Sessão 1. A seguir: Sessão 2 — Árvores, Random Forest e Avaliação.

